

Linux Administration & Application Comprehensive Lab Guide

Linux 管理与应用 — 综合实训指导书

Author: Dawei Yuan

Date: June 2026

Platform: Rocky Linux 9.8 + VirtualBox

Version: v2.1 (Rocky Linux 9.8)

目录

1	项目（实训）名称	4
2	项目（实训）目标	4
3	实训环境准备	4
3.1	推荐平台	4
3.1.1	为什么选 Rocky Linux 9.8	4
3.1.2	下载 ISO 镜像	4
3.1.3	三种 ISO 对比	5
3.2	VirtualBox 安装	5
3.2.1	下载 VirtualBox	5
3.2.2	安装 VirtualBox	5
3.3	VirtualBox 虚拟机配置	5
3.3.1	创建步骤	5
3.3.2	安装建议	6
3.3.3	克隆第二台虚拟机（客户端）	6
3.3.4	安装后初始配置	7
3.4	SSH 远程访问配置	8
3.4.1	启用 SSH 服务	8
3.4.2	网络模式选型	8
3.4.3	双网卡配置步骤	8
3.5	Guest Additions 安装（可选）	9
4	子项目 1：YUM 配置国内源	10
4.1	实验目的	10
4.2	背景说明	10
4.3	操作步骤	10
4.4	验收脚本	12
4.5	验收标准	12
5	子项目 2：搭建 VPN 服务器	13
5.1	实验目的	13
5.2	背景说明	13
5.3	操作步骤	13
5.4	客户端配置	15
5.5	验证	17

5.5.1	服务端验收	17
5.5.2	客户端验收	17
5.6	停止 VPN 服务	17
5.6.1	服务端	18
5.6.2	客户端	18
6	子项目 3: 搭建邮件服务器	19
6.1	实验目的	19
6.2	背景说明	19
6.2.1	获取邮箱授权码	19
6.3	操作步骤	19
6.4	验收脚本	21
6.5	验收标准	21
7	子项目 4: 搭建 DHCP 和 DNS 服务器	22
7.1	实验目的	22
7.2	网卡架构	22
7.3	VirtualBox 操作	22
7.4	操作步骤	22
7.5	DNS 解析原理	24
7.5.1	内网域名解析 (tecmint.lan)	24
7.5.2	外网域名解析 (www.baidu.com)	25
7.5.3	三网卡分工总结	25
7.6	验收脚本	26
7.7	验收标准	26
8	子项目 5: 搭建 Web 服务器	27
8.1	实验目的	27
8.2	操作步骤	27
8.3	验收脚本	29
8.4	验收标准	29
8.5	从 Windows 宿主访问	29
8.5.1	PowerShell / CMD 测试	29
8.5.2	浏览器访问 (需改 hosts)	29
9	子项目 6: 搭建 Samba 服务器	31
9.1	实验目的	31
9.2	业务场景	31
9.3	权限矩阵	31

9.4	操作步骤	31
9.5	验收脚本	35
9.6	验收标准	35
9.7	从 Windows 宿主访问	35
9.7.1	文件资源管理器	35
9.7.2	命令行映射网络驱动器	36
10	子项目 7：搭建代理服务器	37
10.1	实验目的	37
10.2	操作步骤	37
10.3	验收脚本	38
10.4	验收标准	38
11	子项目 8：搭建 NAT 服务器	39
11.1	实验目的	39
11.2	背景说明	39
11.3	前置条件	39
11.4	操作步骤	39
11.5	客户端配置（第二台 VM）	41
11.6	验收脚本	41
11.7	验收标准	42
12	验收脚本使用说明	43
13	辅助教学资料	43

项目（实训）名称

1. YUM 配置国内源
2. 搭建 VPN 服务器
3. 搭建邮件服务器
4. 搭建 DHCP 和 DNS 服务器
5. 搭建 Web 服务器
6. 搭建 Samba 服务器
7. 搭建代理服务器
8. 搭建 NAT 服务器

项目（实训）目标

通过实训，使学生能够完成企业 Linux 服务器的配置、管理与维护。通过实际操作，使学生掌握一定的操作技能，能认真、细致、准确的操作。通过实践过程，培养学生独立思考、独立工作的能力及团队协作精神。

实训环境准备

推荐平台

关键信息 1

本指导书基于 **Rocky Linux 9.8** 编写，与 CentOS Stream 9 完全兼容。推荐使用 VirtualBox 7.x 或 VMware Workstation 17 作为虚拟机平台。

为什么选 Rocky Linux 9.8

- Rocky Linux 是 RHEL 的下游发行版，100% 兼容 RHEL
- 9.8 是 9.x 系列最新稳定版（2026-05-25 发布），软件包较新
- 社区活跃，被广泛用于服务器和开发环境
- 作为 CentOS 的替代品，生态成熟

下载 ISO 镜像

推荐使用中科大镜像源下载：

命令参考

```
https://mirrors.ustc.edu.cn/rocky/9.8/isos/x86_64/Rocky-9.8-
x86_64-boot.iso
```

文件大小约 1.38 GB（Boot ISO），安装时需联网下载所需软件包。

三种 ISO 对比

ISO 类型	文件大小	安装方式	适用场景
Boot ISO	约 1.4 GB	网络安装，按需下载	推荐 —实验、虚拟机测试
DVD ISO	约 14 GB	完整离线安装	无网络环境、批量部署
Minimal ISO	约 2.6 GB	离线最小化安装	纯命令行服务器

VirtualBox 安装**下载 VirtualBox**

来源	下载地址
官方	https://www.virtualbox.org/wiki/Downloads
中科大镜像	https://mirrors.ustc.edu.cn/virtualbox/

选择对应系统的安装包下载。Windows 系统下载文件名类似 VirtualBox-7.0.24-167081-Win.exe

安装 VirtualBox

1. 双击安装包，进入安装向导
2. 按默认选项完成安装（网卡驱动、USB 驱动均勾选）
3. 安装完成后，桌面出现 Oracle VM VirtualBox 图标

VirtualBox 虚拟机配置**创建步骤**

1. VirtualBox → 新建
2. 名称：Rocky-9.8，类型：Linux，版本：Red Hat（64-bit）

配置项	建议值
类型	Linux / Red Hat (64-bit)
内存	2048 MB 以上
CPU	2 核以上
虚拟硬盘	20 GB 动态分配
网络	NAT + Host-Only 双网卡 （外网 + 固定 IP SSH）
显存	128 MB（如需 GUI）

3. 内存：建议 2048 MB

4. 虚拟硬盘：现在创建虚拟硬盘 → VDI → 动态分配 → 20 GB

5. 创建完成后 → 设置 → 存储 → 选择 Boot ISO 镜像

6. 网络设置：**网卡 1：NAT + 网卡 2：Host-Only**（混杂模式均选”拒绝”）

安装建议

- 安装两台虚拟机：一台作为服务器（2GB 内存 +2 核 CPU），一台作为客户端
- 软件选择建议 Minimal Install（纯命令行），体量最小、组件干净
- 安装时在 Installation Destination 选 Automatic 自动分区即可

克隆第二台虚拟机（客户端）

实验 2（VPN）、实验 6（Samba）、实验 8（NAT）需要两台虚拟机协同实验。使用 VirtualBox 克隆功能可快速生成第二台虚拟机，无需重新安装系统。（实验 4 采用单机三网卡自闭环方案，详见子项目 4。）

1. 克隆操作

- (1) 关闭原虚拟机（关机，不是挂起）
- (2) VirtualBox 主界面 → 右键原虚拟机 → **Clone**
- (3) 名称：Rocky-9.8-client，路径默认
- (4) 勾选「重新初始化所有网卡的 MAC 地址」（防止 IP 冲突）
- (5) 克隆类型选 **完全复制**（独立不依赖原盘）

2. 克隆后配置

命令参考

```
# 1. 修改主机名
sudo hostnamectl set-hostname client

# 2. 修复 NAT 网卡 (MAC 地址变化后需重新创建连接)
sudo nmcli connection add type ethernet ifname enp0s8 con-
    name nat autoconnect yes

# 3. 重新生成 SSH 主机密钥 (避免与原机冲突)
sudo rm -f /etc/ssh/ssh_host_*
sudo ssh-keygen -A
sudo systemctl restart sshd
```

3. 验证网络互通

克隆完成后，两台虚拟机的 Host-Only 网卡在同一网段，可互通：

虚拟机	主机名	Host-Only IP	角色
原机器	server	192.168.56.101	承载服务
克隆机	client	192.168.56.102	测试客户端

命令参考

```
# 在两台 VM 上分别查看 IP
ip a | grep 192.168.56

# 互相 ping 测试
ping 192.168.56.101    # 在 client 上执行
ping 192.168.56.102    # 在 server 上执行
```

安装后初始配置

命令参考

```
sudo hostnamectl set-hostname server
yum update -y
yum install -y vim git curl wget net-tools bind-utils
```


SSH 远程访问配置

为方便从 Windows 终端远程操作 Linux 虚拟机，推荐配置 SSH 服务和 NAT + Host-Only 双网卡方案。

启用 SSH 服务

命令参考

```
yum install -y openssh-server
systemctl enable sshd --now
firewall-cmd --permanent --add-service=ssh
firewall-cmd --reload
```

网络模式选型

模式	原理	Windows→ 虚拟机	虚拟机上网
		SSH	
NAT + 双网卡分工		固定 IP	直接连
Host-Only 桥接	接入物理局域网	直接连（WiFi 不	稳定）
NAT 单卡	藏在宿主机 NAT 后	需端口转发	
Host-Only 单卡	虚拟独立局域网	直接连	

关键信息 2

推荐方案：NAT + Host-Only 双网卡
网卡 1 (NAT)：虚拟机上外网（yum install、curl）
网卡 2 (Host-Only)：固定 IP 192.168.56.x，Windows SSH 连接，不受 WiFi 切换影响
Linux 内核自动按目的 IP 选择走哪张网卡，无需手动配置路由。
WiFi 下不推荐桥接模式：WiFi 桥接不支持完整 ARP/DHCP 透传，常导致只获得 IPv6 地址、IPv4 不可用。

双网卡配置步骤

1. 确保虚拟机已关机（不是挂起），进入 VirtualBox → 设置 → 网络
2. 网卡 1：启用网络连接 → 连接方式：**NAT** → 混杂模式：拒绝

- 网卡 2: 启用网络连接 → 连接方式: 仅主机 (Host-Only) 网络 → 混杂模式: 拒绝
- 启动虚拟机, 查看 Host-Only 网卡 IP:

命令参考

```
ip a | grep 192.168.56
```

- 在 Windows 终端 (PowerShell / CMD) 连接:

命令参考

```
ssh 用户名@192.168.56.101
```

注意事项

每个网卡的启用网络连接必须勾选, 混杂模式全部选”拒绝”。
确定按钮变灰通常是虚拟机未关机, 需先关闭电源再进设置。

Guest Additions 安装 (可选)

Guest Additions 提供剪贴板共享、拖拽文件、自适应分辨率等功能。

命令参考

```
yum install -y gcc kernel-devel kernel-headers elfutils-  
libelf-devel  
# VirtualBox 菜单: 设备 → 安装增强功能  
mkdir -p /mnt/cdrom  
mount /dev/cdrom /mnt/cdrom  
cd /mnt/cdrom && ./VBoxLinuxAdditions.run  
reboot
```

重启后通过 VirtualBox 菜单 → 设备 → 共享剪贴板 → 双向, 即可开启复制粘贴。

子项目 1：YUM 配置国内源

实验目的

将 Rocky Linux 9.8 默认的国外 YUM 源替换为阿里云镜像源，提升软件包下载速度。

背景说明

Rocky Linux 的软件包管理工具 YUM 默认配置的是国外官方源，在国内网络环境下往往会导致软件包下载速度缓慢甚至失败。通过将 YUM 源更换为国内主流镜像源，可显著提升软件包下载速度。

操作步骤

1. 修复原有 repo 中的 \$contentdir 变量

Rocky 默认 repo 可能使用 \$contentdir 变量，需先替换为 rocky:

命令参考

```
1 sed -i 's|$contentdir|rocky|g' /etc/yum.repos.d/rocky*.repo
```

2. 备份原有配置（备份目录带日期戳，支持重复执行）

命令参考

```
1 mkdir -p /etc/yum.repos.d/backup-$(date +%Y%m%d)
2 cp /etc/yum.repos.d/*.repo /etc/yum.repos.d/backup-$(date +%Y%m%d)/
```

3. 配置阿里云镜像源

在 /etc/yum.repos.d/ 目录下创建 rocky-aliyun.repo 文件:

命令参考

```
1 cat > /etc/yum.repos.d/rocky-aliyun.repo << 'EOF'
2 [baseos-aliyun]
3 name=Rocky Linux $releasever - BaseOS (Aliyun)
4 baseurl=https://mirrors.aliyun.com/rockylinux/$releasever/BaseOS/$basearch/os/
```

```
5  gpgcheck=1
6  enabled=1
7  gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-Rocky-9
8
9  [appstream-aliyun]
10 name=Rocky Linux $releasever - AppStream (Aliyun)
11 baseurl=https://mirrors.aliyun.com/rockylinux/$releasever/
    AppStream/$basearch/os/
12 gpgcheck=1
13 enabled=1
14 gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-Rocky-9
15
16 [extras-aliyun]
17 name=Rocky Linux $releasever - Extras (Aliyun)
18 baseurl=https://mirrors.aliyun.com/rockylinux/$releasever/
    extras/$basearch/os/
19 gpgcheck=1
20 enabled=1
21 gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-Rocky-9
22 EOF
```

4. 清理缓存并重建

命令参考

```
1  yum clean all
2  yum makecache
```

5. 验证

命令参考

```
1  yum repolist          # 查看启用仓库列表
2  yum search vim        # 测试搜索功能
```

验收脚本

命令参考

```
1 sudo bash labs/lab1_yum_repo/setup.sh    # 一键配置
2 sudo bash labs/lab1_yum_repo/verify.sh   # 验收检查
```

验收标准

- rocky-aliyun.repo 配置文件存在
- yum repolist 显示阿里云仓库已启用
- yum makecache 正常完成
- yum search 可正常使用

子项目 2：搭建 VPN 服务器

实验目的

使用 OpenVPN 搭建 VPN 服务器，实现客户端通过加密隧道安全访问内网资源。

背景说明

虚拟私人网络（Virtual Private Network，简称 VPN）是一种用于连接中、大型企业或团体间私人网络的通讯方法。VPN 的讯息透过公用的网络架构（例如互联网）来传送内网的网络讯息。本次实训使用 OpenVPN 完成 VPN 服务器的搭建。

原理简述：OpenVPN 在服务端和客户端之间建立一条加密的虚拟隧道（TUN 模式，三层 IP 隧道）。双方各创建一个 tun0 虚拟网卡，分配 VPN 子网地址（10.8.0.0/24）。实际数据包经由物理网卡发出前，被 OpenVPN 加密并封装为 UDP 报文；到达对端后解密还原，注入对端 tun0。服务端通过 `push "route"` 将内网路由推送给客户端，使客户端能访问服务端所在的内网资源。

操作步骤

1. 安装 OpenVPN 和 easy-rsa

命令参考

```
1 yum install -y epel-release
2 yum install -y openvpn easy-rsa wget
```

2. 配置 easy-rsa 变量

命令参考

```
1 mkdir -p /opt/easy-rsa && cd /opt/easy-rsa
2 cp -a /usr/share/easy-rsa/3/* ./ 2>/dev/null || cp -a /usr
  /share/easy-rsa/3.*/* ./ 2>/dev/null
3
4 cat > vars << 'EOF'
5 if [ -z "$EASYRSA_CALLER" ]; then
6     echo "You appear to be sourcing an Easy-RSA 'vars'
      file." >&2
7     return 1
8 fi
9 set_var EASYRSA_DN "cn_only"
```

```
10 set_var EASYRSA_REQ_COUNTRY "CN"
11 set_var EASYRSA_REQ_PROVINCE "Beijing"
12 set_var EASYRSA_REQ_CITY "Beijing"
13 set_var EASYRSA_REQ_ORG "example"
14 set_var EASYRSA_REQ_EMAIL "admin@example.com"
15 set_var EASYRSA_NS_SUPPORT "yes"
16 EOF
```

3. 生成证书

命令参考

```
1 ./easyrsa init-pki
2 echo "" | ./easyrsa build-ca nopass
3 echo "" | ./easyrsa gen-req server nopass
4 ./easyrsa --batch sign-req server server
5 openssl ecparam -genkey -name prime256v1 -out /opt/easy-
  rsa/pki/ec.pem
6 echo "" | ./easyrsa gen-req client nopass
7 ./easyrsa --batch sign-req client client
```

4. 服务端配置 (/etc/openvpn/server/server.conf)

命令参考

```
1 mkdir -p /etc/openvpn/server
2 cat > /etc/openvpn/server/server.conf << 'EOF'
3 port 1194
4 proto udp
5 dev tun
6 ca /opt/easy-rsa/pki/ca.crt
7 cert /opt/easy-rsa/pki/issued/server.crt
8 key /opt/easy-rsa/pki/private/server.key
9 dh none
10 ecdh-curve prime256v1
11 server 10.8.0.0 255.255.255.0
12 push "route 192.168.56.0 255.255.255.0"
13 ifconfig-pool-persist ipp.txt
14 keepalive 10 120
```

```
15 max-clients 100
16 status openvpn-status.log
17 log /var/log/openvpn.log
18 verb 3
19 client-to-client
20 persist-key
21 persist-tun
22 duplicate-cn
23 comp-lzo
24 EOF
```

5. 启动服务端（含旧配置清理和证书开放）

命令参考

```
1 # 清理旧配置（支持重复执行）
2 systemctl stop openvpn-server@server 2>/dev/null || true
3 systemctl disable openvpn-server@server 2>/dev/null ||
  true
4 rm -rf /opt/easy-rsa/pki /etc/openvpn/server
5
6 firewall-cmd --permanent --add-port=1194/udp
7 firewall-cmd --reload
8 systemctl enable openvpn-server@server
9 systemctl start openvpn-server@server
10
11 # 开放证书读权限，使客户端可用同名用户 scp 获取
12 chmod -R o+rX /opt/easy-rsa/pki
```

客户端配置

客户端需要从服务端获取 3 个证书文件，然后编写 client.conf 并启动连接。最简便的方式是直接运行客户端脚本（不带参数时会自动扫描 Host-Only 网络发现服务端 192.168.56.101/100/1）：

命令参考

```
1 # 自动发现服务端
2 sudo bash labs/lab2_VPN/client_setup.sh
3 # 或显式指定服务端 IP
```



```
4 sudo bash labs/lab2_VPN/client_setup.sh 192.168.56.101
```

以下手动步骤等价于该脚本所做的工作（脚本自动用 `${SUDO_USER:-$USER}` 检测当前用户名）：

1. 获取证书

命令参考

```
1 # 自动检测当前用户名（兼容 sudo 场景）
2 SSH_USER="${SUDO_USER:-$USER}"
3 SERVER=192.168.56.101
4
5 yum install -y openvpn
6 mkdir -p /etc/openvpn/client
7 scp ${SSH_USER}@${SERVER}:/opt/easy-rsa/pki/ca.crt
   /etc/openvpn/client/
8 scp ${SSH_USER}@${SERVER}:/opt/easy-rsa/pki/issued/client.
   crt /etc/openvpn/client/
9 scp ${SSH_USER}@${SERVER}:/opt/easy-rsa/pki/private/client
   .key /etc/openvpn/client/
```

2. 客户端配置文件（`/etc/openvpn/client/client.conf`）

命令参考

```
1 cat > /etc/openvpn/client/client.conf << EOF
2 client
3 dev tun
4 proto udp
5 remote <server_ip> 1194
6 ca /etc/openvpn/client/ca.crt
7 cert /etc/openvpn/client/client.crt
8 key /etc/openvpn/client/client.key
9 resolv-retry infinite
10 nobind
11 persist-key
12 persist-tun
13 verb 3
14 comp-lzo
```

```
15 pull-filter ignore "route_192.168.56.0"
16 EOF
```

3. 启动客户端

命令参考

```
1 systemctl enable openvpn-client@client
2 systemctl start openvpn-client@client
```

验证

服务端验收

命令参考

```
1 sudo bash labs/lab2_VPN/server_verify.sh
```

- openvpn-server@server 正在运行
- 1194/udp 端口在监听
- tun0 网卡已创建
- server.conf 配置文件存在

客户端验收

命令参考

```
1 sudo bash labs/lab2_VPN/client_verify.sh
```

- openvpn-client@client 正在运行
- tun0 已创建并分配 VPN IP (10.8.0.x)
- 可 ping 通服务端 VPN 地址 10.8.0.1
- 路由表存在 tun0 相关路由，推送已生效

停止 VPN 服务

实验完成后，在两台虚拟机上分别停止 OpenVPN 服务，释放 tun0 虚拟网卡：

服务端

命令参考

```
1 sudo systemctl stop openvpn-server@server
2 sudo systemctl disable openvpn-server@server
```

客户端

命令参考

```
1 sudo systemctl stop openvpn-client@client
2 sudo systemctl disable openvpn-client@client
```

停止后 ip a 中 tun0 消失，Host-Only 网络恢复正常。

子项目 3：搭建邮件服务器

实验目的

使用 Postfix 搭建 SMTP 邮件发送客户端，通过 QQ 邮箱或 163 邮箱的 SMTP 服务器中继实现邮件发送功能。

背景说明

本实验并非搭建完整公网邮件服务器。完整邮件系统需 Postfix + Dovecot + 自有域名 + MX DNS 记录。本实验中的 Postfix 仅作为 SMTP 中继客户端，借用 QQ 或 163 邮箱的 SMTP 服务器代理发送邮件。脚本会根据邮箱后缀（@qq.com / @163.com）自动选择对应的 SMTP 服务器。

工作流程：Postfix 接收本地邮件 → SASL 认证登录 SMTP 服务器（需邮箱地址 + 授权码）→ 邮件服务商验证身份后以该邮箱名义将邮件投递到最终收件人。

Postfix 作为工业级 MTA，通过纯文本配置文件即可操控完整 SMTP 协议行为：relayhost 指定中继目标、smtp_use_tls 控制传输加密、smtp_sasl_auth_enable 启用身份认证。

获取邮箱授权码

脚本运行时需输入发件邮箱地址和对应授权码。目前支持两种邮箱：

提供商	SMTP 服务器: 端口	授权码获取方式
QQ 邮箱	smtp.qq.com:587	登录 mail.qq.com → 设置 → 账户 → POP3/SMTP 服务 → 开启
163 邮箱	smtp.163.com:465	登录 mail.163.com → 设置 → POP3/SMTP/IMAP → 开启

两种邮箱授权码获取方式类似：开启 SMTP 服务后，按提示发送短信验证，即可获得授权码。

配置文件 → 程序解析 → 协议行为变化这一范式贯穿全部 8 个实验，是本课程的核心教学模式。

操作步骤

1. 安装 Postfix

命令参考

```
1 yum install -y postfix s-nail cyrus-sasl-plain
```

2. 配置 Postfix main.cf

在 `/etc/postfix/main.cf` 末尾添加 (`$SMTP_HOST` 和 `$SMTP_PORT` 由脚本自动检测, 标记行用于支持重复执行时不产生重复配置):

命令参考

```

1 # 清理旧配置
2 sed -i '/^#_=====lab3_begin_=====$/,/^#_=====lab3_end_
   =====$/d' /etc/postfix/main.cf
3 sed -i '/^relayhost[[:space:]]*=/d;_/^smtp_tls_wrappermode
   [[:space:]]*=/d;_/^smtp_generic_maps[[:space:]]*=/d' /
   etc/postfix/main.cf
4 cat >> /etc/postfix/main.cf << EOF
5 # ===== lab3 begin =====
6 relayhost = [$SMTP_HOST]:$SMTP_PORT
7 smtp_sasl_auth_enable = yes
8 smtp_sasl_password_maps = hash:/etc/postfix/sasl_passwd
9 smtp_sasl_security_options = noanonymous
10 smtp_use_tls = yes
11 smtp_tls_security_level = encrypt
12 smtp_tls_CAfile = /etc/pki/tls/certs/ca-bundle.crt
13 smtp_generic_maps = hash:/etc/postfix/generic
14 smtp_tls_wrappermode = yes
15 # ===== lab3 end =====
16 EOF

```

3. 创建认证文件和发件人地址映射

命令参考

```

1 echo "[$SMTP_HOST]:$SMTP_PORT_${SENDER_EMAIL}:${AUTH_CODE}
   " > /etc/postfix/sasl_passwd
2 chmod 600 /etc/postfix/sasl_passwd
3 postmap /etc/postfix/sasl_passwd
4
5 cat > /etc/postfix/generic << EOF
6 root@$(hostname) ${SENDER_EMAIL}
7 root                ${SENDER_EMAIL}
8 EOF
9 postmap /etc/postfix/generic

```

`smtp_generic_maps` 强制将所有本地发件人地址（如 `root`）改写为授权邮箱，满足 163 服务器 Mail from must equal authorized user 的要求。

4. 启动并测试

命令参考

```
1 systemctl enable postfix
2 systemctl restart postfix
3 echo "Test_mail_from_Rocky_Linux" | mail -s "Test_Subject"
  recipient@qq.com
```

5. 问题排查

如果收不到邮件，查看日志：

命令参考

```
1 tail -f /var/log/maillog
```

验收脚本

命令参考

```
1 sudo bash labs/lab3_email/setup.sh      # 会提示输入 QQ 邮箱和
    授权码
2 sudo bash labs/lab3_email/verify.sh
```

验收标准

- postfix 服务正在运行
- main.cf 中 relayhost 已配置
- /etc/postfix/sasl_passwd 和.db 文件存在
- postfix check 语法检查通过
- 发送测试邮件成功（默认发至 nucleocore2026@126.com，可自定义）

子项目 4：搭建 DHCP 和 DNS 服务器

实验目的

使用 dnsmasq 一站式搭建 DHCP 和 DNS 服务器，实现局域网 IP 地址动态分配和域名解析。本实验采用 **单机三网卡自闭环方案**：给 server VM 新增第三张内部网络网卡（NIC3），dnsmasq 绑定内网 IP，同一台 VM 通过 dhclient 向自己的 dnsmasq 获取 DHCP 地址，实现 DHCP + DNS 全流程验证，**无需第二台虚拟机**。

网卡架构

网卡	连接方式	IP 来源	用途
NIC1 (enp0s3)	NAT	VirtualBox DHCP	访问外网（yum install、 DNS 转发）
NIC2 (enp0s8)	Host-Only	静态 192.168.56.x	Windows SSH 连接
NIC3 (enp0s9, 新增)	内部网络	静态 10.100.0.1/24	dnsmasq DHCP + DNS 验证

注意事项

NIC3 为内部网络（Internal Network）模式，不是 Host-Only。内部网络与 NIC1/NIC2 完全隔离，不存在 VirtualBox 内置 DHCP 冲突。

VirtualBox 操作

1. 关闭 VM（关机，不是挂起）
2. VirtualBox → 设置 → 网络 → 网卡 3 → 启用网络连接
3. 连接方式选择：内部网络 (Internal Network)
4. 名称填写：intnet-dhcp（可自定义）
5. 启动 VM

操作步骤

1. 安装 dnsmasq

命令参考

```
1 yum install -y dnsmasq
```

2. 配置 NIC3 内部网络网卡

setup.sh 自动检测未配置 IP 的网卡，按顺序尝试 enp0s9、enp0s10、enp0s16、ens9、ens10。通过 nmcli 创建连接 dhcp-intnet，分配静态 IP 10.100.0.1/24：

命令参考

```
1 # enp0s9 为脚本自动探测到的 NIC3（也可能是 enp0s10/ens9  
   等，脚本内用变量 $NIC3）  
2 nmcli connection add type ethernet ifname enp0s9 con-name  
   dhcp-intnet \  
3     ip4 10.100.0.1/24 autoconnect yes
```

3. 配置 dnsmasq

编辑 /etc/dnsmasq.conf：

命令参考

```
1 # DNS 配置  
2 listen-address=10.100.0.1  
3 bind-interfaces  
4 domain=tecmin.tlan  
5 server=223.5.5.5          # 阿里 DNS（外网转发）  
6 server=223.6.6.6          # 阿里 DNS（外网转发）  
7 address=/tecmin.tlan/10.100.0.1  
8 cache-size=150  
9  
10 # DHCP 配置  
11 dhcp-range=10.100.0.50,10.100.0.150,12h  
12 dhcp-option=3,10.100.0.1  
13 dhcp-option=6,10.100.0.1  
14 dhcp-leasefile=/var/lib/misc/dnsmasq.leases
```


关键信息 3

`listen-address` 只绑定内网 IP 10.100.0.1，配合 `bind-interfaces` 确保 `dnsmasq` 仅监听 NIC3，不干扰 NIC1/NIC2 的正常网络通信。DHCP 地址池 10.100.0.50--150 与 Host-Only 网段（192.168.56.x）完全隔离。

4. 配置防火墙

命令参考

```
1 firewall-cmd --permanent --add-service=dns
2 firewall-cmd --permanent --add-service=dhcp
3 firewall-cmd --reload
```

5. 启动服务并验证 DHCP 自获取

命令参考

```
1 systemctl enable dnsmasq
2 systemctl restart dnsmasq
3
4 # NIC3 通过 dhclient 向自己发起 DHCP 请求
5 dhclient enp0s9
6
7 # 检查是否获取到 IP（应落在 10.100.0.50--150）
8 ip addr show enp0s9 | grep 10.100.0
```

6. 验证 DNS

命令参考

```
1 nslookup tecmint.lan 10.100.0.1      # 本地域名解析
2 nslookup www.baidu.com 10.100.0.1    # 外网 DNS 转发
```

DNS 解析原理

`dnsmasq` 是一个轻量级网络服务程序，一个二进制、一套配置文件就同时提供 DNS 和 DHCP 两项功能。理解它的 DNS 解析流程对掌握本实验至关重要。

内网域名解析（tecmint.lan）

用户执行 `ping tecmint.lan`，系统通过 `/etc/resolv.conf` 指向的 DNS 服务器 10.100.0.1 发起查询。`dnsmasq` 收到请求后的查找顺序：

- (1) 查本地缓存——之前访问过则直接返回
- (2) 查 `/etc/hosts` ——有则返回
- (3) 匹配 `dnsmasq.conf` 中的 `address=/tecmint.lan/10.100.0.1` ——命中!返回 10.100.0.1

整个过程在 `enp0s9`（内部网络）内完成，无需访问外网。

外网域名解析（`www.baidu.com`）

用户 `nslookup www.baidu.com 10.100.0.1` 的完整流程：

- (1) DNS 请求通过 `enp0s9` 到达 `dnsmasq`（监听 10.100.0.1）
- (2) `dnsmasq` 查本地缓存 → 未命中 → 查 `/etc/hosts` → 未命中
- (3) 匹配 `dnsmasq.conf` 规则：
 - `address=/tecmint.lan/10.100.0.1` ——不匹配 `www.baidu.com`，跳过
 - `server=223.5.5.5` ——命中！转发给上游阿里 DNS
- (4) Linux 内核路由决策：223.5.5.5 不在任何本地网段，走默认路由
- (5) 出口网卡为 `enp0s8` (NAT)，VirtualBox NAT 引擎执行 SNAT 后发往互联网
- (6) 阿里 DNS 返回 `www.baidu.com` 的 IP 地址，原路返回 → `enp0s8` → `dnsmasq` → 客户端

关键信息 4

DNS 解析完成后，后续的实际通信（如 `ping` 的 ICMP 包、`curl` 的 HTTP 请求）直接走内核路由从 `enp0s8` (NAT) 出去，与 `dnsmasq` 无关。`dnsmasq` 只负责域名 → IP 的翻译工作。

三网卡分工总结

网卡	角色	在本实验中的分工
<code>enp0s9</code> （内部网络）	DNS/DHCP 入口	接收客户端 DNS 查询 + DHCP 请求
<code>enp0s8</code> (NAT)	外网出口	DNS 向上游转发（访问 223.5.5.5）+ 用户实际外网流量
<code>enp0s3</code> (Host-Only)	SSH 管理	Windows 远程登录，不参与实验流程

验收脚本

命令参考

```
1 sudo bash labs/lab4_DHCP_DNS/setup.sh
2 sudo bash labs/lab4_DHCP_DNS/verify.sh
```

关键信息 5

`setup.sh` 自动完成 NIC3 发现、静态 IP 配置、dnsmasq 安装配置、dhclient 验证全流程。

验收标准

- dnsmasq 服务正在运行
- DHCP 地址池已配置
- DNS 域名 tecmint.lan 已配置
- 内部网络网卡（NIC3）已配置 IP 10.100.0.1
- DHCP 租约文件 `/var/lib/misc/dnsmasq.leases` 有记录（NIC3 成功获取 IP）
- nslookup 可解析 tecmint.lan
- 配置语法正确

子项目 5：搭建 Web 服务器

实验目的

使用 Apache (httpd) 搭建 Web 服务器，实现基于域名的虚拟主机，使一个 IP 地址承载多个网站。

操作步骤

1. 安装 Apache 和 BIND

命令参考

```
1 yum install -y httpd bind
```

2. 创建虚拟主机目录和默认页面

命令参考

```
1 mkdir -p /var/www/test-web1 /var/www/test-web2
2 echo "this is web1" > /var/www/test-web1/index.html
3 echo "this is web2" > /var/www/test-web2/index.html
```

3. 配置 DNS 区域解析

在 /etc/named.conf 末尾添加两个 zone 定义：

命令参考

```
1 zone "test-web1.com" IN { type master; file "web1.com.zone
    "; };
2 zone "test-web2.com" IN { type master; file "web2.com.zone
    "; };
```

创建区域文件 /var/named/web1.com.zone (web2 同理，仅域名替换为 test-web2)，将 www 的 A 记录指向服务器 IP：

命令参考

```
1 $TTL 1D
2 @      IN SOA  @ rname.invalid. (
3                               0      ; serial
```

```

4          1D      ; refresh
5          1H      ; retry
6          1W      ; expire
7          3H )    ; minimum
8 @          IN NS  www.test-web1.com.
9 www        IN A    192.168.56.101

```

4. 配置虚拟主机

创建 /etc/httpd/conf.d/vhosts.conf:

命令参考

```

1 <VirtualHost *:80>
2     ServerAdmin admin@test-web1.com
3     DocumentRoot /var/www/test-web1
4     ServerName www.test-web1.com
5     DirectoryIndex index.html
6     ErrorLog logs/test-web1.error_log
7     CustomLog logs/test-web1.access_log common
8 </VirtualHost>
9
10 <VirtualHost *:80>
11     ServerAdmin admin@test-web2.com
12     DocumentRoot /var/www/test-web2
13     ServerName www.test-web2.com
14     DirectoryIndex index.html
15     ErrorLog logs/test-web2.error_log
16     CustomLog logs/test-web2.access_log common
17 </VirtualHost>

```

5. 启动服务并验证

命令参考

```

1 systemctl enable named httpd
2 systemctl restart named httpd
3
4 # 测试虚拟主机
5 curl -H "Host: www.test-web1.com" http://127.0.0.1/

```

```
6 curl -H "Host: www.test-web2.com" http://127.0.0.1/
```

验收脚本

命令参考

```
1 sudo bash labs/lab5_Web/setup.sh
2 sudo bash labs/lab5_Web/verify.sh
```

验收标准

- httpd 和 named 服务正在运行
- curl test-web1.com 返回 “this is web1”
- curl test-web2.com 返回 “this is web2”
- 虚拟主机配置文件存在

从 Windows 宿主访问

VM 的 Host-Only 网卡与 Windows 宿主互通（同一网段 192.168.56.0/24），Apache 监听 *:80，firewalld 已放行 http 服务，可直接从 Windows 浏览器访问。

PowerShell / CMD 测试

命令参考

```
curl -H "Host: www.test-web1.com" http://192.168.56.101/
curl -H "Host: www.test-web2.com" http://192.168.56.101/
```

浏览器访问（需改 hosts）

编辑 C:\Windows\System32\drivers\etc\hosts（管理员权限），添加：

命令参考

```
192.168.56.101 www.test-web1.com
192.168.56.101 www.test-web2.com
```

保存后在浏览器直接访问 <http://www.test-web1.com> 和 <http://www.test-web2.com>。浏览器请求 HTTP 时会自动写 Host 头，Apache 根据所在 <VirtualHost> 的 ServerName 返回对应网页。

关键信息 6

Apache 虚拟主机利用 HTTP 请求头中的 `Host` 字段区分来访域名，即使两个域名解析到同一个 IP 也能返回不同网页，这是 lab5 的核心。

子项目 6：搭建 Samba 服务器

实验目的

使用 Samba 搭建跨平台文件共享服务器，实现多用户、多组、分权限的文件共享方案。

业务场景

该公司分支机构有 system、develop、productdesign 和 test 共 4 个小组。个人办公机操作系统为 Windows/Linux，服务器操作系统为 Rocky Linux 9.8。需要设计一套安全文件共享方案。

权限矩阵

目录	所有者	develop	productdesign	test	develop_test
/data/share	system	—	—	—	—
/data/share/develop	develop	读写	不可访问	不可访问	—
/data/share/productdesign	productdesign	不可访问	读写	不可访问	—
/data/share/test	test	不可访问	不可访问	读写	—
/data/share/library	system	只读	只读	只读	—
/data/share/develop_test	—	读写	不可访问	读写	读写
/data/share/temp	system	读写	读写	读写	—

表 1: Samba 共享权限矩阵

操作步骤

1. 安装 Samba

命令参考

```
1 yum install -y samba samba-client samba-common cifs-utils
```

2. 创建目录结构

命令参考

```
1 mkdir -p /data/share/{develop,productdesign,test,library,
  develop_test,temp}
```


3. 创建用户和组

命令参考

```
1 groupadd -f system
2 groupadd -f develop
3 groupadd -f productdesign
4 groupadd -f test
5 groupadd -f develop_test
6
7 useradd -g develop -G develop_test -d /data/share/develop
   -s /sbin/nologin develop
8 useradd -g productdesign -G develop_test -d /data/share/
   productdesign -s /sbin/nologin productdesign
9 useradd -g test -G develop_test -d /data/share/test -s /
   sbin/nologin test
10 useradd -g system -G develop,productdesign,test,
   develop_test -d /data/share -s /sbin/nologin system
```

4. 设置目录权限

命令参考

```
1 chmod 755 /data/share
2 chown system:system /data/share
3 chmod 2770 /data/share/develop /data/share/productdesign /
   data/share/test /data/share/develop_test
4 chmod 3777 /data/share/temp
5 chmod 755 /data/share/library
6
7 chown develop:system /data/share/develop
8 chown productdesign:system /data/share/productdesign
9 chown test:system /data/share/test
10 chown system:system /data/share/library
11 chown system:system /data/share/temp
12
13 setfacl -m g:develop:rx /data/share/develop_test
14 setfacl -m g:test:rx /data/share/develop_test
```

5. SELinux 放行 Samba 访问

Rocky Linux 9 默认启用 SELinux 强制模式,Samba 仅允许访问系统标记为 `samba_share_t` 的路径。自定义目录 `/data/share` 需要手动添加上下文标记:

命令参考

```
1 semanage fcontext -a -t samba_share_t "/data/share(/.*)?"
2 restorecon -Rv /data/share
```

注意事项

未配置此行将导致 Windows 无法访问共享目录。症状: 文件资源管理器提示“网络错误”、凭据正确但拒绝访问。常见于 Rocky Linux 9 / RHEL 9 系列等 SELinux 默认 enforcing 的系统。

6. 配置 `/etc/samba/smb.conf`

命令参考

```
1 [global]
2     workgroup = SYSTEM
3     server string = Linux Samba Server
4     security = user
5     map to guest = Bad User
6
7 [system]
8     comment = System Admin Share
9     path = /data/share/
10    writeable = yes
11    valid users = system
12    browseable = yes
13
14 [library]
15    path = /data/share/library
16    writeable = no
17    browseable = yes
18    guest ok = yes
19
20 [temp]
21    path = /data/share/temp
22    writeable = yes
23    browseable = yes
```

```
24     guest ok = yes
25
26 [develop]
27     path = /data/share/develop
28     writeable = yes
29     browseable = yes
30     valid users = @develop,@system
31
32 [productdesign]
33     path = /data/share/productdesign
34     writeable = yes
35     browseable = yes
36     valid users = @productdesign,@system
37
38 [test]
39     path = /data/share/test
40     writeable = yes
41     browseable = yes
42     valid users = @test,@system
43
44 [develop_test]
45     path = /data/share/develop_test
46     writeable = yes
47     browseable = yes
48     valid users = @develop,@test,@system
```

7. 添加 Samba 用户

命令参考

```
1  smbpasswd -a system      # 输入密码
2  smbpasswd -a develop    # 输入密码
3  smbpasswd -a productdesign # 输入密码
4  smbpasswd -a test       # 输入密码
5
6  testparm -s             # 测试配置
7  systemctl enable smb --now
```

验收脚本

命令参考

```
1 sudo bash labs/lab6_Samba/setup.sh
2 sudo bash labs/lab6_Samba/verify.sh
```

注意事项

setup.sh 中所有 Samba 用户默认密码为 123456。正式环境中请修改。

验收标准

- smb 服务正在运行
- testparm 配置语法正确
- 7 个共享均已定义
- 各用户目录存在且权限正确
- develop 用户可访问 develop 共享，被拒绝访问 productdesign 共享

从 Windows 宿主访问

setup.sh 已配置 firewall-cmd --add-service=samba 放行 139/445 端口，安装完成后可从 Windows 文件资源管理器直接挂载 Samba 共享。

文件资源管理器

地址栏输入：

命令参考

```
\\192.168.56.101\develop
```

在弹出的认证窗口中输入用户名 develop 和密码 123456,即可浏览 /data/share/develop 目录。

不同账号登录结果不同：

- develop 可访问 \\192.168.56.101\develop，不可访问 \\...\productdesign
- productdesign 可访问 \\...\productdesign，不可访问 \\...\develop
- system（管理员）可访问全部共享

命令行映射网络驱动器

命令参考

```
net use Z: \\192.168.56.101\develop /user:develop 123456
```

子项目 7：搭建代理服务器

实验目的

使用 Squid 搭建 HTTP 正向代理服务器，实现客户端通过代理访问外部网络。

操作步骤

1. 安装 Squid

命令参考

```
1 yum install -y squid
```

2. 配置代理

编辑 `/etc/squid/squid.conf`，设置代理端口 8080 并允许内网访问：

命令参考

```
1 acl localnet src 0.0.0.1-0.255.255.255
2 acl localnet src 10.0.0.0/8
3 acl localnet src 172.16.0.0/12
4 acl localnet src 192.168.0.0/16
5 acl localnet src fc00::/7
6 acl localnet src fe80::/10
7
8 acl SSL_ports port 443
9 acl Safe_ports port 80
10 acl Safe_ports port 21
11 acl Safe_ports port 443
12 acl Safe_ports port 70
13 acl Safe_ports port 210
14 acl Safe_ports port 1025-65535
15 acl Safe_ports port 280
16 acl Safe_ports port 488
17 acl Safe_ports port 591
18 acl Safe_ports port 777
19
20 http_access deny !Safe_ports
21 http_access deny CONNECT !SSL_ports
22 http_access allow localhost manager
```

```
23 http_access deny manager
24 http_access allow localnet
25 http_access allow localhost
26
27 http_port 8080
28
29 coredump_dir /var/spool/squid
30
31 refresh_pattern ^ftp:          1440      20%      10080
32 refresh_pattern ^gopher:       1440      0%       1440
33 refresh_pattern -i (/cgi-bin/|\?) 0       0%        0
34 refresh_pattern .               0        20%     4320
```

3. 启动并测试

命令参考

```
1 systemctl enable squid --now
2
3 # 通过代理访问网页
4 curl -x http://localhost:8080 http://www.baidu.com
```

4. 客户端使用

在客户端的浏览器或终端中配置代理地址 `http://< 服务器 IP>:8080` 即可。

验收脚本

命令参考

```
1 sudo bash labs/lab7_proxy/setup.sh
2 sudo bash labs/lab7_proxy/verify.sh
```

验收标准

- squid 服务正在运行
- 8080 端口在监听
- `curl -x` 通过代理可访问外网（HTTP 200/301/302）

子项目 8：搭建 NAT 服务器

实验目的

使用 iptables 配置 NAT（网络地址转换）服务器，使内网主机通过该服务器访问外网，同时配置防火墙规则限制访问。

背景说明

NAT（Network Address Translation，网络地址转换）用于在本地网络使用私有地址、连接互联网时转换为全局 IP 地址的技术。本次实训使用 iptables 作为 NAT 服务器接入网络。

前置条件

服务器需要至少两张网卡：一张外网（NAT，连互联网）、一张内网（Host-Only，连客户端）。脚本自动识别：有默认路由的网卡作为外网 (WAN)，另一张作为内网 (LAN)，无需手动指定 eth0/eth1。

操作步骤

1. 安装 iptables-services

命令参考

```
1 yum install -y iptables-services
```

2. 启用 IP 转发

命令参考

```
1 echo "net.ipv4.ip_forward=1" >> /etc/sysctl.conf
2 sysctl -w net.ipv4.ip_forward=1
```

3. 停止 firewalld，启用 iptables

命令参考

```
1 systemctl stop firewalld
2 systemctl disable firewalld
3 systemctl enable iptables --now
```


4. 配置 NAT 规则

命令参考

```
1 # 自动识别网卡：有默认路由的为外网(WAN)，另一张为内网(LAN)
2 WAN_IF=$(ip route show default | awk '{print $5; exit}')
3 LAN_IF=$(ip -o link show | awk -F': ' -v w="$WAN_IF" '$2
   != "lo" && $2 != w {print $2; exit}')
4
5 # 清空现有规则
6 iptables -F; iptables -t nat -F; iptables -X; iptables -t
   nat -X
7
8 # 默认策略
9 iptables -P INPUT DROP
10 iptables -P FORWARD DROP
11 iptables -P OUTPUT ACCEPT
12
13 # 允许回环、已建立连接、SSH、ICMP
14 iptables -A INPUT -i lo -j ACCEPT
15 iptables -A OUTPUT -o lo -j ACCEPT
16 iptables -A INPUT -m state --state ESTABLISHED,RELATED -j
   ACCEPT
17 iptables -A FORWARD -m state --state ESTABLISHED,RELATED -
   j ACCEPT
18 iptables -A INPUT -p tcp --dport 22 -j ACCEPT
19 iptables -A INPUT -p icmp --icmp-type echo-request -j
   ACCEPT
20
21 # NAT 转发：内网→外网
22 iptables -A FORWARD -i "$LAN_IF" -o "$WAN_IF" -j ACCEPT
23
24 # 外网→内网只放行 Web、DNS、邮件服务
25 iptables -A FORWARD -i "$WAN_IF" -o "$LAN_IF" -p tcp --
   dport 80 -j ACCEPT
26 iptables -A FORWARD -i "$WAN_IF" -o "$LAN_IF" -p tcp --
   dport 443 -j ACCEPT
27 iptables -A FORWARD -i "$WAN_IF" -o "$LAN_IF" -p udp --
   dport 53 -j ACCEPT
28 iptables -A FORWARD -i "$WAN_IF" -o "$LAN_IF" -p tcp --
```

```
        dport 25 -j ACCEPT
29 iptables -A FORWARD -i "$WAN_IF" -o "$LAN_IF" -p tcp --
        dport 110 -j ACCEPT
30
31 # SNAT 地址伪装
32 iptables -t nat -A POSTROUTING -o "$WAN_IF" -j MASQUERADE
33
34 # 保存规则
35 iptables-save > /etc/sysconfig/iptables
```

5. 验证规则

命令参考

```
1 iptables -L -v -n          # 查看 filter 表
2 iptables -t nat -L -v -n   # 查看 nat 表
```

客户端配置（第二台 VM）

NAT 实验需要两台虚拟机：一台做 NAT 网关（运行 `server_setup.sh`），另一台做内网客户端。客户端只需一张 Host-Only 网卡（请在 VirtualBox 中禁用其自带的 NAT 网卡，避免绕过网关），并将默认网关指向 NAT 服务器的 Host-Only 地址。`client_setup.sh` 默认保留客户端现有 IP，仅用 `nmcli` 设置默认网关与 DNS（如需更改 IP 才传第 2 个参数，且经 SSH 运行时提示确认以避免断连）。脚本末尾自动进行端到端验证（默认路由、ping 网关、ping 8.8.8.8、curl 网页），输出与 `verify.sh` 相同的 [PASS]/[FAIL] 格式；其中“ping 网关”仅为参考（网关可能丢弃 ICMP），能否出网才是判据。

验收脚本

命令参考

```
1 # 服务器(网关)端:
2 sudo bash labs/lab8_NAT/server_setup.sh
3 sudo bash labs/lab8_NAT/verify.sh
4
5 # 客户端（第二台 VM，参数为服务器的 Host-Only IP）:
6 sudo bash labs/lab8_NAT/client_setup.sh 192.168.56.101
```

验收标准

- `net.ipv4.ip_forward = 1`
- `iptables` 服务正在运行
- FORWARD 链存在 ACCEPT 规则
- POSTROUTING 链存在 MASQUERADE 规则
- 规则已持久化到 `/etc/sysconfig/iptables`
- `firewalld` 已停止（避免冲突）

验收脚本使用说明

每个实验目录下均提供了两个脚本：

文件	作用
<code>setup.sh</code>	一键安装配置脚本，学生执行以完成实验
<code>verify.sh</code>	验收检查脚本，教师用于检查学生实验是否正确完成

注意事项

例外：实验 2（VPN）与实验 8（NAT）为双机实验，安装脚本按角色拆分为 `server_setup.sh` 与 `client_setup.sh`；实验 2 的验收脚本相应为 `server_verify.sh` / `client_verify.sh`（无单一 `verify.sh`）。

所有 `verify.sh` 输出统一格式：

命令参考

```
[PASS] 检查项描述
[FAIL] 检查项描述
[WARN] 检查项描述
=== 结果：N 通过，M 失败 ===
```

- 全部 PASS → exit 0（验收通过）
- 有 FAIL → exit 1（验收不通过）

批量检查所有实验：

命令参考

```
1 for lab in labs/lab*; do
2     echo "===_Checking_$(basename_$lab)_==="
3     sudo bash "$lab/verify.sh" || echo "FAILED"
4     echo ""
5 done
```

辅助教学资料

- <http://www.linux.org> —The Linux Home Page at Linux Online

- <http://bbs.chinaunix.net> —中国最大的 Linux/Unix 技术社区
- <https://docs.rockylinux.org> —Rocky Linux 官方文档